

RDBMS Architecture



Application

E
n
g
i
n
e

Query Processor (Optimizer/Executor)
(DP/Histograms/Sampling/PlanDiagrams/PlanBouquet)

Indexes
(B-Trees/Hashing/Bitmaps)

Buffer Manager
(LRU-2)

Concurrency Control
(Locking/Isolation)

Recovery
(ARIES)

P
l
a
t
f
o
r
m

Operating System

Hardware
[Processors, Memory, Disks]

Transaction Concurrency Control

E0 261

Jayant Haritsa
Indian Institute of Science



ACID Properties of Transactions

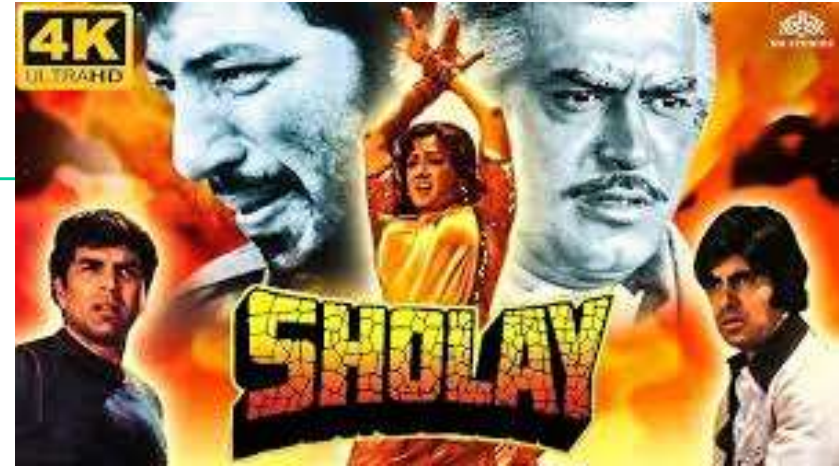
- Atomicity:
 - all or nothing
- Consistency
 - from one consistent state to another
- Isolation
 - as if executed alone in the system
- Durability
 - results permanent after commit



TRANSACTIONS in BOLLYWOOD



Movie Scene



- Kidnappers Gabbar and Sambha have captured Basanti, the apple of Veeru's eye (after dancing round so many trees, his eyesight isn't good).
- To release Basanti, they demand Veeru hand over a ransom of “पच्चीस लाख” (kidnappers know Veeru is an IISc software engineer).

Movie Scene (contd)

- Veeru is told to show up at Chowpatty beach at “ठीक बारह बजे” for the exchange, failing which Basanti’s “लाश मिलेगी”.
- Veeru, who had been saving the money for his mother’s heart operation, complies after saying the obligatory “माँ” !



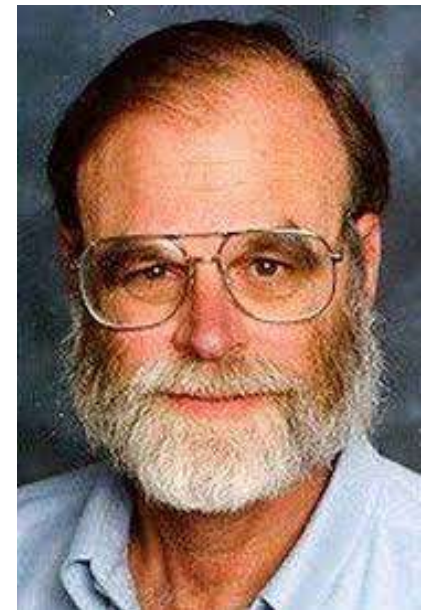
Transactions in the Movies

- Atomicity: Veeru has either Basanti or money
- Consistency: No fake money (or fake Basanti!)
- Isolation: No hidden camera, please!
- Durability: Veeru and Basanti walk off into the sunset and live happily ever after ...



Today's Paper

- Granularity of Locks and Degrees of Consistency in a Shared Data Base
 - J. N. Gray, R. A. Lorie, G. R. Putzolu, I. L. Traiger
- Modeling in DBMS, 1976
- Turing award in 1998 for Jim Gray
- Missing at sea (Jan 2007)



Lock Modes



Motivation: A “Simple” Protocol

- Lock modes:
 - mode **S** for shared and **X** for exclusive access
 - compatibility: $(S,S) = T$, otherwise **F**
- Unit:
 - a relation
- Behavior:
 - Two-phase locking (2PL) for Read locks
 - Strict two-phase locking for Write locks



Locking Granularity

- Fine-grained units for increasing concurrency
- Coarse-grained units for decreasing overheads
- Solution:
 - hierarchical lockable units:
 - DB, areas, files, pages, tuples, attributes
- Correctness problem:
 - T1 S.locks a tuple, T2 X.locks the file?
 - solution?
 - create a collision path



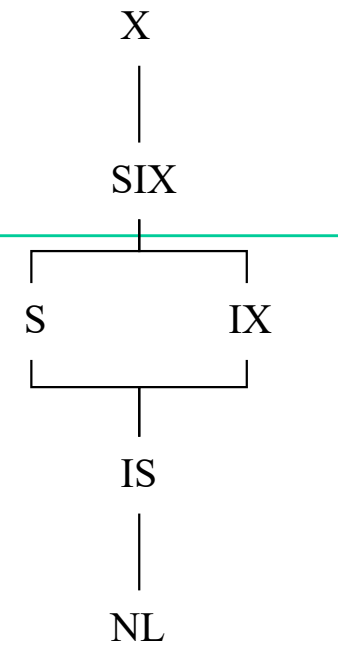
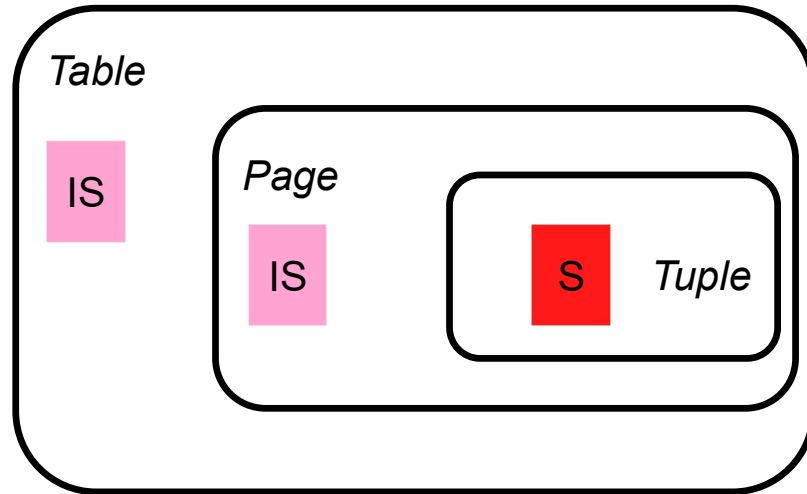
Locking Protocol

- Database: as hierarchy of lockable units
- Locking: to lock a unit
 - first lock all *containing* units with “intention”
 - intention locks: **IS**, **IX** (intention to upgrade)
 - no implicit locking of sub-tree unlike **S** and **X**
 - **SIX**: implicit locking in **S** and explicit in **X** mode
- Unlocking:
 - release all relevant locks simultaneously at EOT, or leaf to root

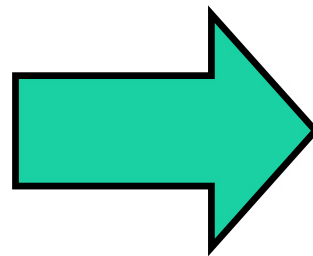


Intention Locks (IS/IX/SIX)

Hierarchy

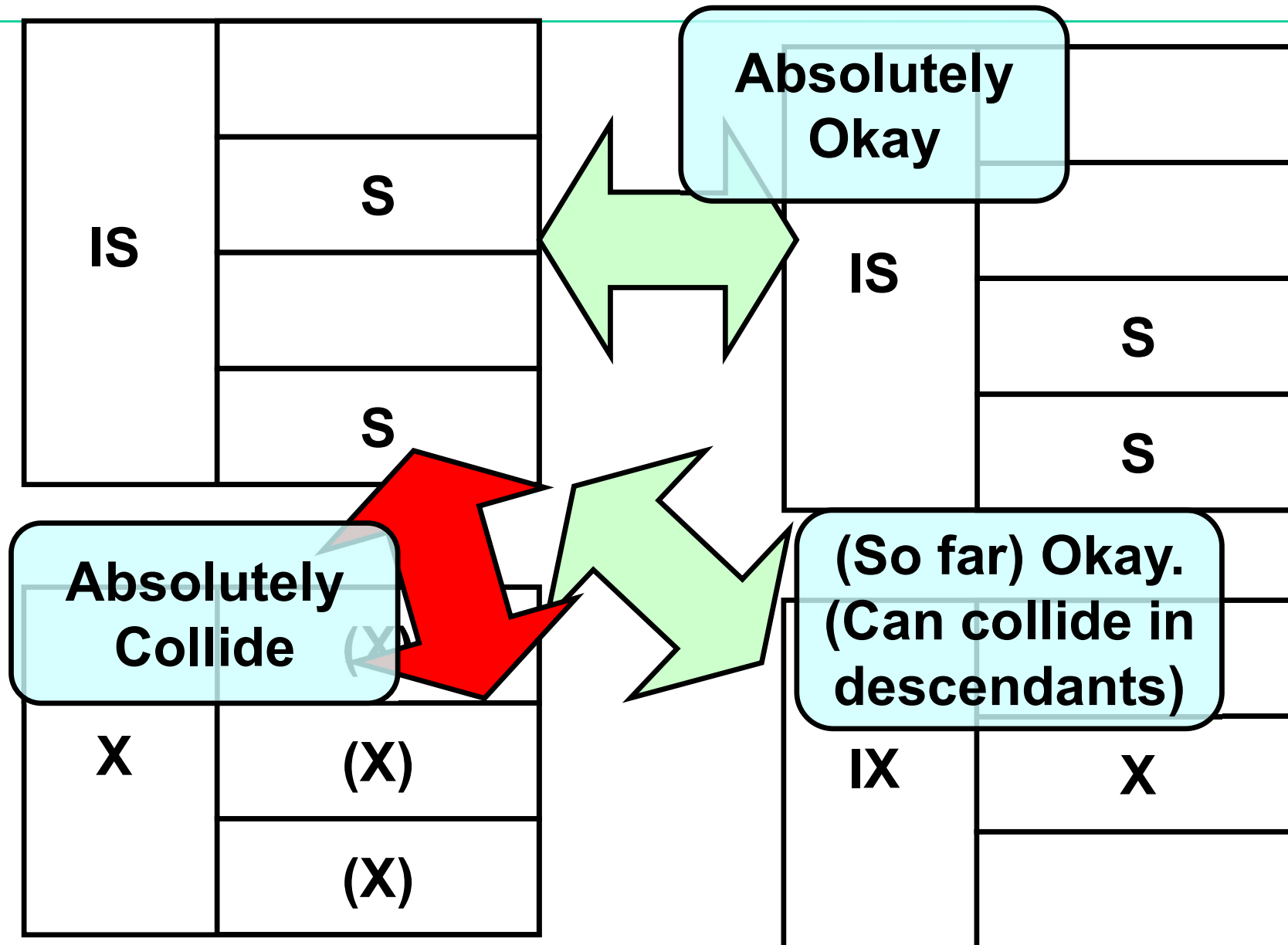


	S	X
S	Y	N
X	N	N

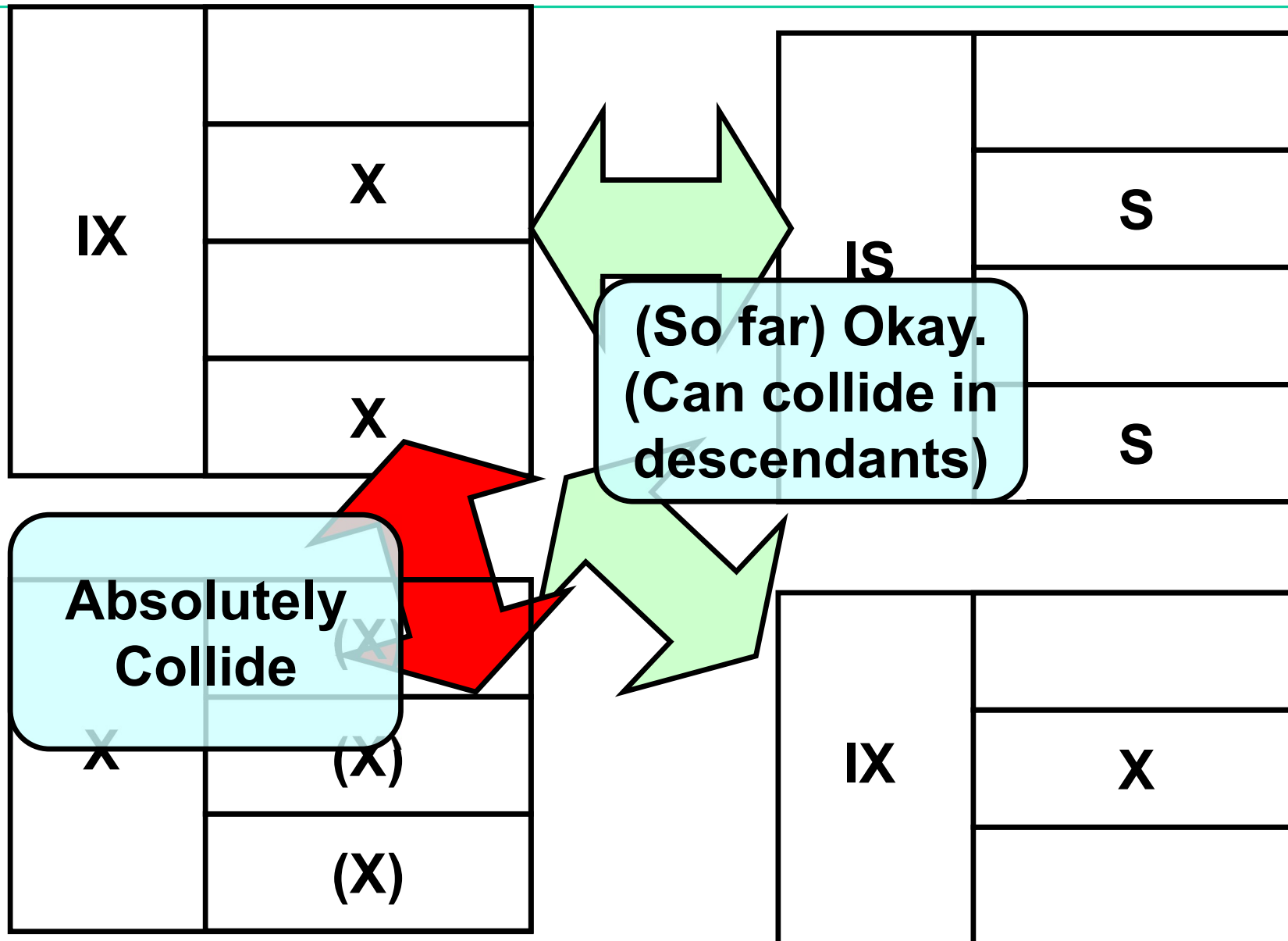


	IS	S	IX	SIX	X
IS	Y	Y	Y	Y	N
S	Y	Y	N	N	N
IX	Y	N	Y	N	N
SIX	Y	N	N	N	N
X	N	N	N	N	N

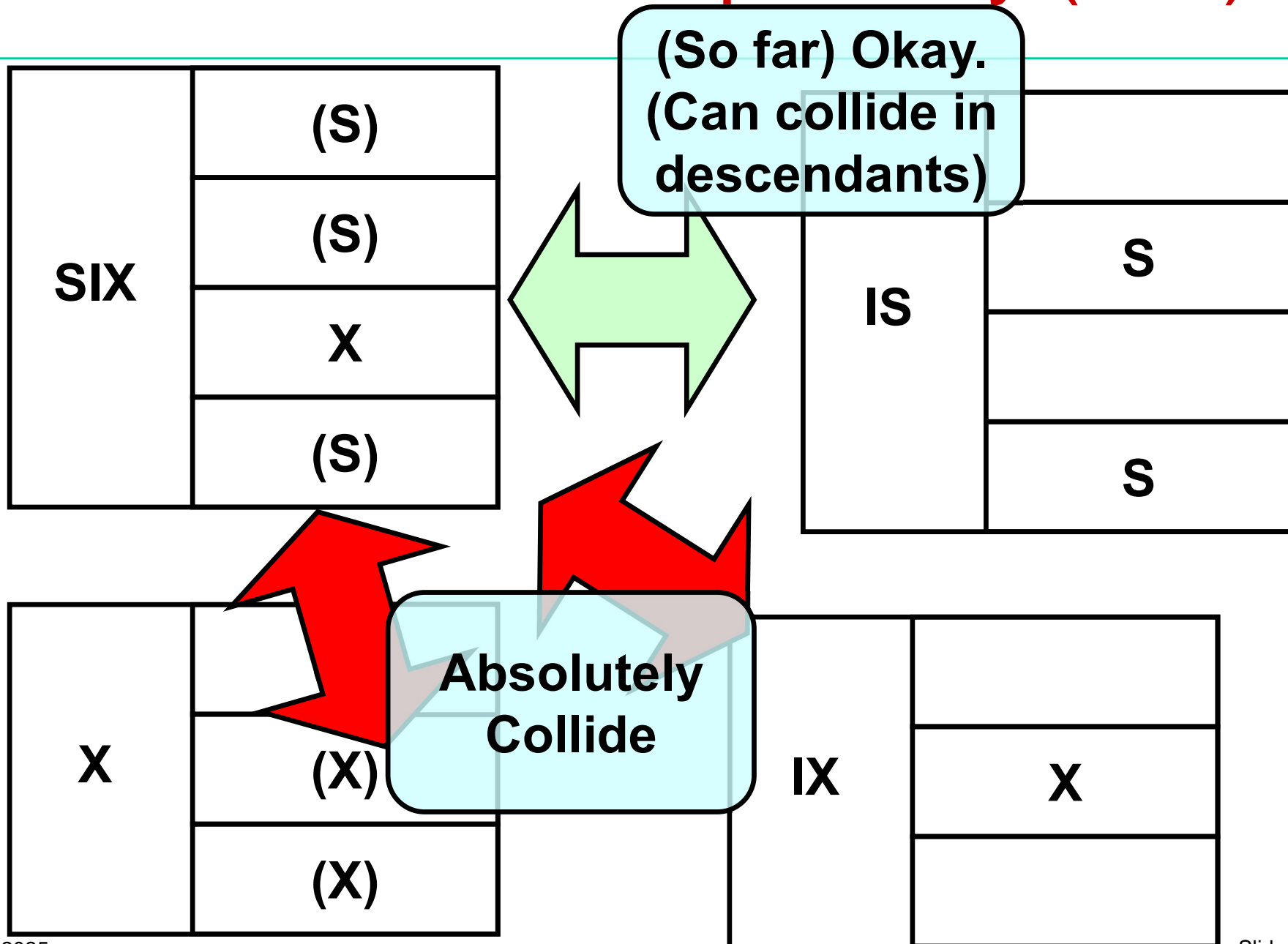
Intention Lock Compatibility (IS)



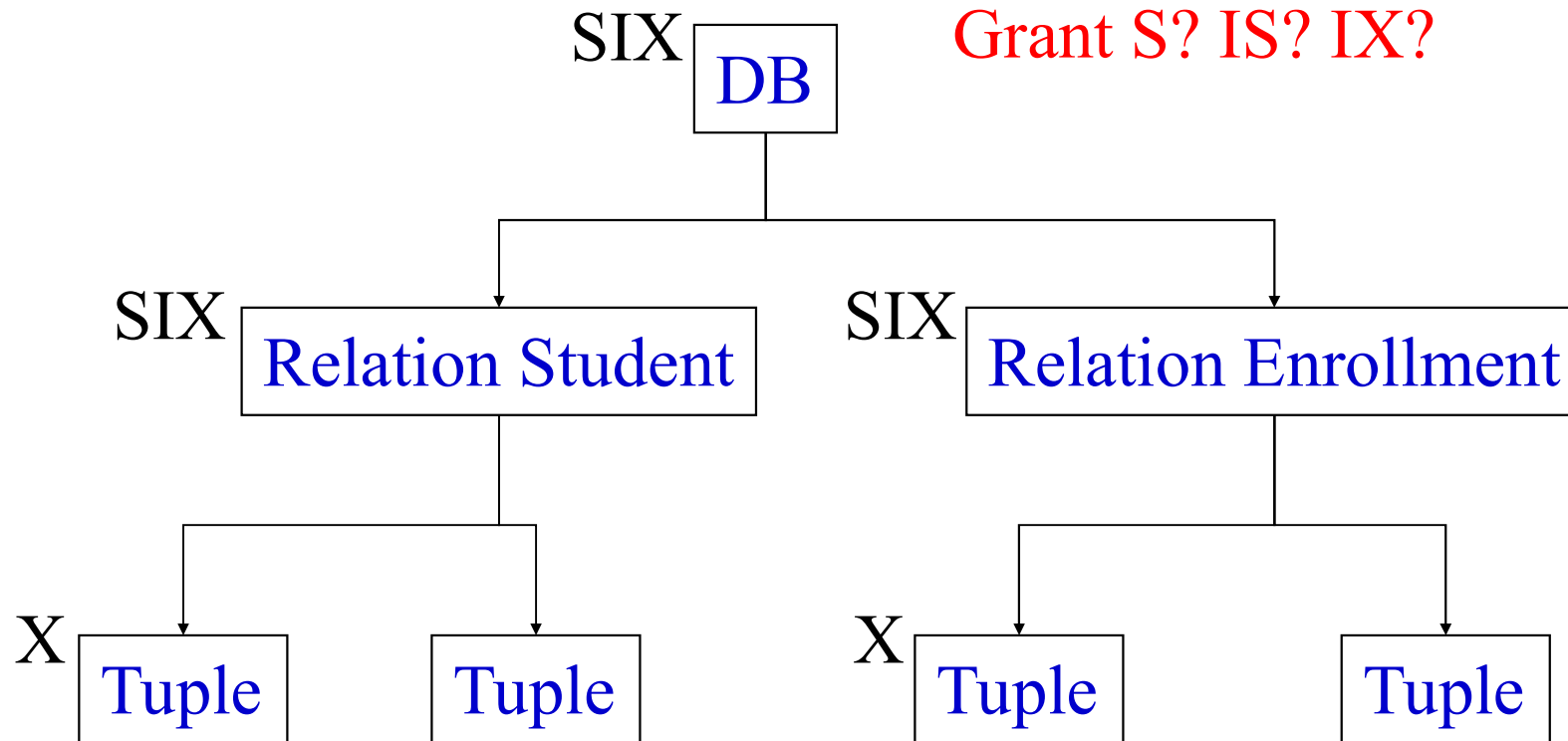
Intention Lock Compatibility (IX)



Intention Lock Compatibility (SIX)



Compatibility Examples



- (SIX, S) = No (SIX, IS) = Yes (SIX, IX) = No

Locking Protocol: DAG

- DAG of units:
 - S lock at least one path to the node
 - X lock all paths to the node

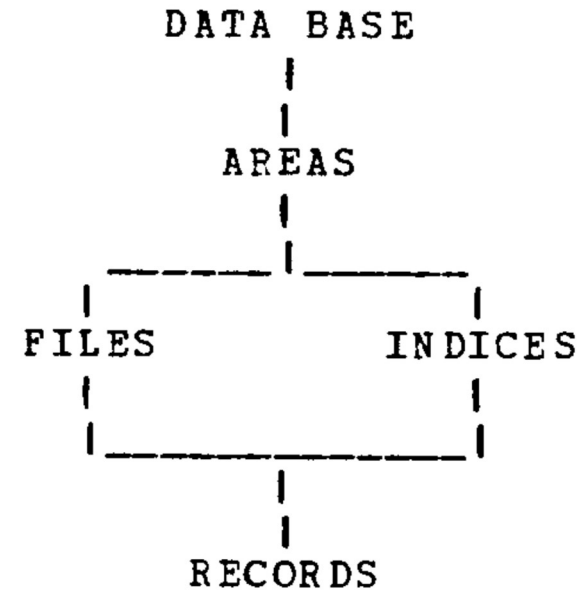


Figure 3. A non-hierarchical lock graph.

- Implicit S if one parent is S, SIX, X
- Implicit X if all parents are X

Additional Lock Modes

- Based on semantics of updates
 - increment / decrement locks (as in assignment!)



Transaction-level Consistency



Motivation for Consistency Degrees

Pablo Diego José Francisco de Paula Juan Nepomuceno María de los Remedios Cipriano de la Santísima Trinidad Ruiz y Picasso

- Garbage values
 - T1: A = 10; w(A)
 - T2: A = 25; w(A)
 - **Problem:** Final value may be neither 10 nor 25!
 - **Solution: Short write locks (latches), Degree 0 consistency**
- Lost updates
 - T1: r(A) w(A) commit
 - T2: w(A) commit
 - **Problem:** T2's update is lost
 - **Solution: Long write locks, Degree 1 consistency**
- Dirty Reads:
 - T1: w(A) abort
 - T2: r(A)
 - **Problem:** T2 has read a non-existent value
 - **Solution: Long write locks, Short Read locks, Degree 2 consistency**
- Inconsistent Reads
 - T1: w(A) commit
 - T2: r(A) r(A)
 - **Problem:** Different committed values read by T2 for same data object
 - **Solution: Long write locks, Long read locks, Degree 3 consistency**

Phantom Tuples

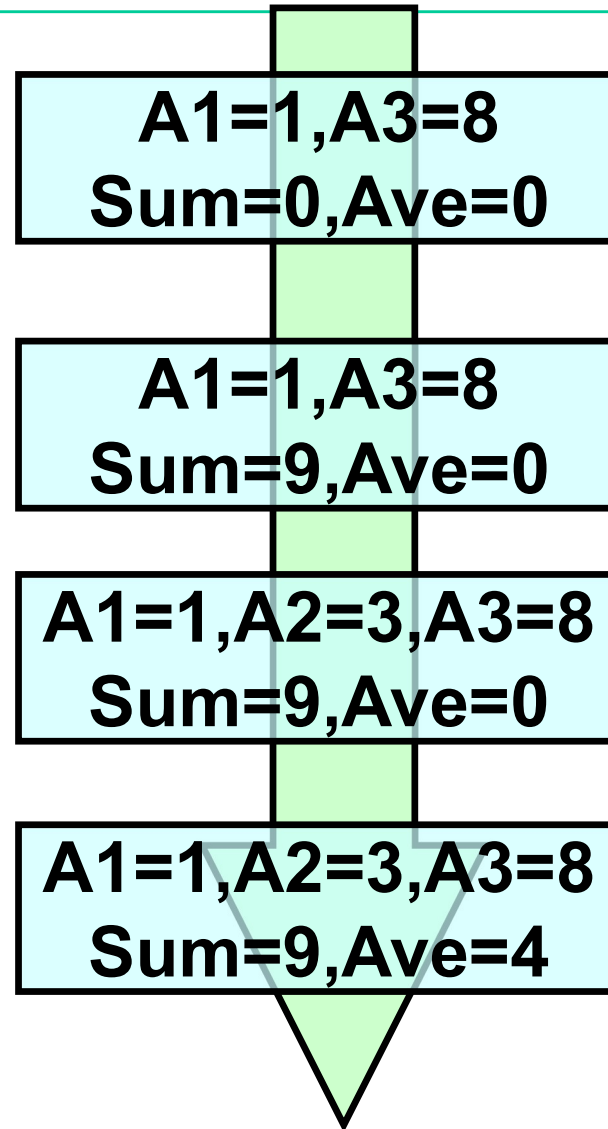


Initial:
A1=1,A3=8

Transaction1

Insert
A2=3
COMMIT

Serialized Result
Sum=9, Ave=4.5
or
Sum=12, Ave=4



Transaction2

Sum
A1→A3

Ave
A1→A3

Locks in Fully Serializable Mode

- Same Locks as Repeatable Read
- .. and Special Locks to Prevent Phantoms
 - Predicate Lock (Semantic Lock)
 - Efficiently checkable for simple predicates
 - Boolean combination of Attr op const where op is $<$, $=$, $\neq$, $>$
 - Bank_branch = "SBI IISc Campus"
 - NP-hard for arbitrary predicates
 - Key Range Lock
 - Select avg(gpa) from Student where Age between 20 and 30
 - Lock index keys between 20 and 30
 - Worst case, lock whole table



Additional Issues

- Deadlock Handling
 - Simplest solution is **timeout!**
 - Alternatively, cycle checking
- Deadlock Prevention
 - Use other CC methods such as Timestamping and Validation



CC Support in Practice

IBM DB2:

- Optional explicit locking (by user) at table level
 - LOCK TABLE students in SHARE/EXCLUSIVE MODE
 - locks held until and automatically released at EOT
- Automatic implicit locking (by system) with “isolation level” setting
SQL transaction isolation levels:
 - Serializable
 - set long write / long read / phantom locks
 - repeated read
 - set long write / long read locks, thus phantom insert possible
 - read committed
 - set long write/short read locks
 - read uncommitted
 - set long write/no read locks(somewhat different names in DB2)
- For more information, see Jim Gray’s well-known book “Transaction Processing”



END Concurrency Control

